

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №4
«Професійне ООП та система типів Python»
з курсу «Професійний Python»
Варіант №1 — «Система управління студентами»**

Виконав: Лис Артур

Група: ФЕП-31

Перевірів(ла): _____

Дата: 08.09.2026

Львів — 2026

1. Мета роботи

Побудувати типізовану об'єктну модель предметної області «облік студентів» так, як це роблять у професійному Python-кодi: сутності як класи з інкапсульованим станом, value objects як frozen dataclass, наслідування лише там, де є справжнє is-a, композиція для всього іншого, абстракції через ABC і Protocol, dependency injection замість жорстких залежностей, generic-репозиторій на TypeVar, TypedDict на межі із зовнішнім світом і статична перевірка всього цього через туру у strict-режимі. Побічна мета — закласти шар, на який у наступних роботах ляжуть база даних і REST API.

2. Індивідуальний варіант і постановка задачі

Варіант №1 — «Система управління студентами». Domain objects: Student, Group, Course, Teacher, Grade. Потрібно реалізувати enrollment, додавання оцінки, середній бал, academic status, repository, reporting service та notification abstraction.

Обов'язковий мінімум методичних вказівок і де він закритий у кодi:

Вимога	Реалізація в пакеті domain
не менше 5 domain classes	Entity, Person, Student, Teacher, Course, Group, GradeBook — 7
щонайменше 2 dataclass	PersonName, GroupCode, Grade, Credits, Container — 5
frozen value object	усі чотири value objects: frozen=True, slots=True
property	Grade.ects, Student.average, Student.group, GradeBook.earned_credits, Repository.kind...
inheritance	Entity → Person → Student / Teacher; ThresholdPolicy → StrictPolicy
composition	Student має GradeBook, Course має Teacher, Group має Student-ів, GradeImporter має parser/validator/formatter
abstract base class	Repository[T], AcademicStatusPolicy, BaseGradeImporter
Protocol	HasId, Serializable (runtime_checkable), Notifier, LineParser, GradeValidator, RowFormatter
щонайменше 3 dunder methods	__eq__, __hash__, __repr__, __str__, __len__, __iter__, __contains__, __getitem__, __lt__, __add__

dependency injection	усі сервіси приймають залежності в <code>__init__</code> ; <code>Container.build()</code> — composition root
3 принципи SOLID	усі п'ять, розділ 12
type hints для public API	усюди; мпуу <code>--strict</code> без зауважень на 26 файлах
TypeVar	<code>T = TypeVar("T", bound=HasId); Points = TypeVar("Points", int, float)</code>
generic class	<code>Repository[T], InMemoryRepository[T]</code>
TypedDict	<code>StudentPayload</code> (з <code>NotRequired</code>), <code>GradePayload</code> , <code>StudentReport</code> , <code>GroupReport</code>
мпуу або Pyright	мпуу 1.18, <code>[tool.mypy] strict = true</code> у <code>pyproject.toml</code>
application/service layer	<code>services.py</code> : <code>EnrollmentService</code> , <code>GradingService</code> , <code>ReportingService</code>
демонстраційний сценарій	<code>python -m student_manager.domain (__main__.py)</code>

Із завдань підвищеної складності виконано: `parameterized generic repository`, `bound TypeVar`, `constrained TypeVar`, `runtime_checkable Protocol`, `custom collection class` із `__getitem__`, `__iter__`, `__contains__`, `arithmetic dunder` (`Credits.__add__`), `total_ordering` для `Grade`, `class factory` (`Student.from_payload`, `Container.build`), `strategy pattern` (`AcademicStatusPolicy`), `repository abstraction`, `immutable value objects`, `custom exception hierarchy`, `interface segregation` через кілька `Protocols`, `ручний dependency injection container` і `CI-подібна команда перевірки типів`.

3. Предметна область і структура проєкту

3.1. Опис предметної області та список domain entities

Деканат веде облік студентів. Викладачі читають курси з певною кількістю кредитів ECTS. Студенти належать до академічних груп, записуються на курси і отримують оцінки за 100-бальною шкалою, які переводяться в літери ECTS. Від системи потрібні: середній бал студента, його `academic status` (відмінник, встигає, ризик відрахування), рейтинг, звіт по групах і повідомлення студентам про зарахування, запис на курс та незадовільні оцінки.

Поділ на сутності та `value objects` я робив за одним запитанням: чи має об'єкт ідентичність? Студентка Гнатишин лишається тією самою, навіть якщо змінить прізвище — це сутність з `id`. А оцінка 95 — та сама оцінка, де б вона не стояла; у неї нема «хто», лише «яка» — це `value object`.

Клас	Вид	Роль
Entity	базова сутність	id, рівність і hash за id
Person	сутність	спільне для людей: ім'я; батько для Student і Teacher
Student	сутність	група, e-mail, власна залікова книжка (GradeBook)
Teacher	сутність	кафедра; на нього посилається Course
Course	сутність	назва, кредити, викладач
Group	сутність-колекція	код групи і список студентів; середній бал групи
GradeBook	колекція	курси, на які записано студента, та оцінки з кожного
PersonName	value object	ім'я та прізвище, нормалізовані
GroupCode	value object	назва групи, перевірена за шаблоном ЛР1
Grade	value object	бал 0..100, літера ECTS, порівняння
Credits	value object	кредити ECTS, підтримує +
AcademicStatus	enum	без оцінок / відмінник / встигає / ризик відрахування

3.2. Структура проєкту

Модель живе в підпакеті `student_manager.domain`, файли якого повторюють рекомендовану структуру методичних вказівок (`value_objects`, `models`, `protocols`, `repositories`, `services`, `dto`) плюс кілька власних: `errors`, `notifications`, `container`, `importing`. Демонстрація запускається командою `python -m student_manager.domain`. Репозиторій: <https://github.com/ArturLys/student-manager>, гілка `lab4`.

```

student_manager/
├── src/
│   └── student_manager/
│       ├── analytics/
│       │   ├── __init__.py
│       │   ├── __main__.py
│       │   ├── analytics.py
│       │   ├── benchmark.py
│       │   └── data.py

```

```

├── decorators.py
├── processors.py
├── domain/
│   ├── __init__.py
│   ├── __main__.py
│   ├── container.py
│   ├── dto.py
│   ├── errors.py
│   ├── importing.py
│   ├── models.py
│   ├── notifications.py
│   ├── protocols.py
│   ├── repositories.py
│   ├── services.py
│   └── value_objects.py
├── __init__.py
├── cli.py
├── config.py
├── exceptions.py
├── main.py
├── models.py
├── registry.py
├── tests/
│   ├── test_analytics.py
│   ├── test_domain.py
│   └── test_registry.py
├── pyproject.toml
├── README.md
└── .gitignore

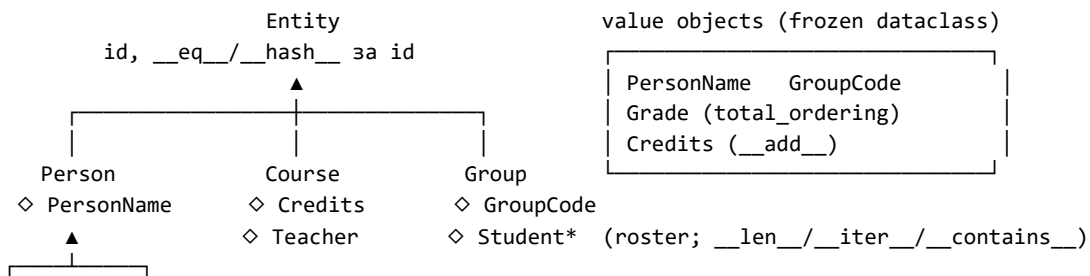
```

3.3. Зв'язок із попередніми роботами

Dataclass Student з ЛР1 (models.py) і StudentRegistry я навмисно не чіпав: це мінімальна модель, на якій тримається консольне меню, і вона своє завдання виконує. Пакет domain — її професійний наступник: та сама предметна область, але з ідентичністю сутностей, value objects, абстракціями сховища та сервісним шаром. Обидва шари поділяють config.py (межі шкали, шаблон групи) та базовий виняток StudentManagerError з exceptions.py, тож меню ЛР1 зможе перехопити будь-яку помилку нової моделі одним ехсепт. Демонстраційні дані — ті самі дев'ять студентів і 36 оцінок, що в пакеті analytics ЛР2, тому середні бали й рейтинг збігаються з попереднім звітом.

4. Class diagram

Стрілка ▲ — наслідування (is-a), ромб ◇ — композиція/агрегація (has-a), пунктир — залежність через абстракцію. Value objects показано окремо, бо вони не мають зв'язків між собою.



```

Student      Teacher
◇ GroupCode
◇ GradeBook — {Course: [Grade]}  (__len__/_iter__/_contains__/_getitem__)

```

```

AcademicStatusPolicy (ABC)      Repository[T: HasId] (ABC, Generic)
      ▲                               ▲
      |                               |
ThresholdPolicy  StrictPolicy —super()→ InMemoryRepository[T]

```

```

Protocols (structural): HasId · Serializable (runtime_checkable) · Notifier
                        LineParser · GradeValidator · RowFormatter  (importing.py)

```

```

EnrollmentService ----> Repository[Student], Repository[Course], Repository[Group], Notifier
GradingService       ----> Repository[Student], Repository[Course], Notifier, AcademicStatusPolicy
ReportingService     ----> Repository[Student], Repository[Group], AcademicStatusPolicy
Container.build()    --> єдине місце, де абстракції отримують конкретні класи

```

```

TypedDict на межі: StudentPayload ⇔ Student (from_payload / to_dict), GradePayload,
                  StudentReport, GroupReport (результати ReportingService)

```

5. Value objects: dataclass, frozen, properties, dunder methods

Усі чотири value objects — `@dataclass(frozen=True, slots=True)`. `frozen` дає незмінність і `hash`, тож `Grade` можна класти в `set` і використовувати як ключ; `slots` — компактність і заборону випадкових атрибутів. Валідація в `__post_init__`: об'єкт із балом 101 або порожнім прізвищем просто не створюється. Нормалізацію (обрізання пробілів) у `frozen`-класі доводиться робити через `object.__setattr__` — звичайне присвоєння `self.first = ...` кине `FrozenInstanceError`, і це документований ідіом, а не хак.

`Grade` показує кілька речей одразу: `property` `ects` обчислює літеру з балу, `__lt__` написано вручну, а `functools.total_ordering` домальовує `<=`, `>`, `>=` (рівність генерує `dataclass`). `Credits.__add__` — арифметичний `dunder`, завдяки якому `sum(course.credits ..., start=ZERO_CREDITS)` дає сумарні кредити студента без жодного циклу. `validated_points` — функція з `constrained TypeVar`: приймає `int` або `float` і повертає значення того самого типу, тож `type checker` знає, що з `int` на вході буде `int` на виході.

```

"""Value objects: immutable, compared by value, validated once in __post_init__.
```

```

A Grade of 95 is the same grade wherever it appears, so it has no identity – that is what makes it a
value object rather than an entity. frozen=True gives immutability and hashability, slots=True keeps
the instances small and forbids accidental new attributes.
"""

```

```

from __future__ import annotations

import re
from collections.abc import Mapping
from dataclasses import dataclass
from enum import StrEnum
from functools import total_ordering
from typing import TypeVar

```

```

from student_manager.config import EXCELLENT_THRESHOLD, GRADE_MAX, GRADE_MIN, GROUP_PATTERN
from student_manager.exceptions import InvalidGradeError, InvalidGroupError

PASSING_THRESHOLD = 60.0

# Constrained TypeVar: the function accepts int OR float and promises to return the same one.
Points = TypeVar("Points", int, float)

def validated_points(value: Points) -> Points:
    """Range check on the 100-point scale that hands the value back with the type it came with."""
    if not GRADE_MIN <= value <= GRADE_MAX:
        raise InvalidGradeError(value)
    return value

class AcademicStatus(StrEnum):
    """Result of an academic status policy; a str subclass, so it serialises as plain text."""

    NO_GRADES = "без оцінок"
    EXCELLENT = "відмінник"
    GOOD = "встигає"
    AT_RISK = "ризик відрахування"

@dataclass(frozen=True, slots=True)
class PersonName:
    """First and last name, normalised and never empty."""

    first: str
    last: str

    def __post_init__(self) -> None:
        # A frozen dataclass blocks plain assignment; object.__setattr__ is the documented way
        # to normalise fields inside __post_init__.
        object.__setattr__(self, "first", self.first.strip())
        object.__setattr__(self, "last", self.last.strip())
        if not self.first or not self.last:
            raise ValueError("ім'я та прізвище не можуть бути порожніми")

    @property
    def full(self) -> str:
        return f"{self.last} {self.first}"

    @property
    def initials(self) -> str:
        return f"{self.last} {self.first[0]}."

    def __str__(self) -> str:
        return self.full

```

```

@dataclass(frozen=True, slots=True)
class GroupCode:
    """Academic group name such as ФЕН-31c; validated against the lab-1 pattern."""

    value: str

    def __post_init__(self) -> None:
        object.__setattr__(self, "value", self.value.strip())
        if not re.fullmatch(GROUP_PATTERN, self.value):
            raise InvalidGroupError(self.value)

    @property
    def faculty(self) -> str:
        """Letters before the dash: ФЕН."""
        return self.value.split("-", 1)[0]

    @property
    def stream(self) -> str:
        """Digits after the dash: 31."""
        return re.sub(r"\D", "", self.value.split("-", 1)[1])

    def __str__(self) -> str:
        return self.value

@total_ordering
@dataclass(frozen=True, slots=True)
class Grade:
    """One mark on the 100-point scale with its ECTS letter.

    dataclass generates __eq__; __lt__ is written by hand and total_ordering derives <=, >, >=.
    """

    points: float

    def __post_init__(self) -> None:
        object.__setattr__(self, "points", float(validated_points(self.points)))

    @property
    def ects(self) -> str:
        """ECTS letter by the LNU scale."""
        for floor, letter in ((90, "A"), (81, "B"), (71, "C"), (61, "D"), (PASSING_THRESHOLD, "E")):
            if self.points >= floor:
                return letter
        return "FX"

    @property
    def is_passing(self) -> bool:
        return self.points >= PASSING_THRESHOLD

    @property
    def is_excellent(self) -> bool:
        return self.points >= EXCELLENT_THRESHOLD

```

```

def to_dict(self) -> Mapping[str, object]:
    return {"points": self.points, "ects": self.ects}

def __lt__(self, other: Grade) -> bool:
    return self.points < other.points

def __str__(self) -> str:
    return f"{self.points:.1f} ({self.ects})"

@dataclass(frozen=True, slots=True)
class Credits:
    """ECTS credits of a course; supports + so that sum() over courses works."""

    ects: int

    def __post_init__(self) -> None:
        if self.ects < 0:
            raise ValueError("кількість кредитів не може бути від'ємною")

    def __add__(self, other: Credits) -> Credits:
        return Credits(self.ects + other.ects)

    def __str__(self) -> str:
        return f"{self.ects} кред."

ZERO_CREDITS = Credits(0)

```

6. Сутності: inheritance, super(), encapsulation, composition

6.1. Чому сутності — не dataclass

dataclass генерує `__eq__` за всіма полями. Для value object це правильно, для сутності — ні: `Student(id=1, «Гнатишин»)` і `Student(id=1, «Гнатишин-Ковальчук»)` мають бути одним студентом. Тому сутності написані вручну: `Entity` тримає `id` за property без setter-a, порівнює й хешує лише за `id` і типом, а `__eq__` повертає `NotImplemented` для чужих типів, щоб Python міг спробувати порівняння з іншого боку. `__slots__` у кожному класі ієрархії — інкапсуляція на рівні мови: помилка в назві атрибута стає `AttributeError`, а не тихим сміттям в об'єкті.

```

class Entity:
    """Base for anything with identity. Equality and hash come from the id, not from the fields."""

    __slots__ = ("_id",)

    def __init__(self, entity_id: EntityId) -> None:
        if entity_id <= 0:
            raise ValueError(f"id має бути додатним, отримано {entity_id!r}")
        self._id = entity_id

```

```

@property
def id(self) -> EntityId:
    return self._id

def __eq__(self, other: object) -> bool:
    if not isinstance(other, Entity):
        return NotImplemented
    return type(self) is type(other) and self._id == other._id

def __hash__(self) -> int:
    return hash((type(self).__name__, self._id))

def __repr__(self) -> str:
    return f'{type(self).__name__}(id={self._id})'

```

6.2. Inheritance та super()

Єдина ієрархія, де наслідування чесне: Student і Teacher справді є Person — обидва мають id та ім'я, обидва друкуються прізвищем. Кожен `__init__` через `super()` віддає батькові його частину роботи (Entity перевіряє id, Person зберігає ім'я) і додає лише своє. `__str__` теж будується ланцюжком: Teacher бере рядок Person і дописує кафедру, Student — групу.

```

class Person(Entity):
    """Someone with a name. Student and Teacher *are* persons, so inheritance is the honest choice."""

    __slots__ = ("name",)

    def __init__(self, entity_id: EntityId, name: PersonName) -> None:
        super().__init__(entity_id)
        self.name = name

    def __str__(self) -> str:
        return self.name.full

    def __repr__(self) -> str:
        return f'{type(self).__name__}(id={self.id}, name={self.name.full!r})'

class Teacher(Person):
    """A lecturer; courses refer to him, he does not own them."""

    __slots__ = ("department",)

    def __init__(self, entity_id: EntityId, name: PersonName, department: str) -> None:
        super().__init__(entity_id, name)
        self.department = department.strip()

    def __str__(self) -> str:
        return f'{super().__str__()} ({self.department})'

```

6.3. Composition: Course має Teacher, Student має GradeBook

Course не успадковує нічого, крім Entity: він *має* викладача і *має* кредити — це has-a. to_dict() робить Course структурно сумісним із Protocol Serializable (розділ 8), хоча Course про цей Protocol не знає.

```
class Course(Entity):
    """A discipline with credits and a teacher. Course *has* a teacher – composition, not
    inheritance."""

    __slots__ = ("title", "credits", "teacher")

    def __init__(self, entity_id: EntityId, title: str, credits: Credits, teacher: Teacher) -> None:
        super().__init__(entity_id)
        self.title = title.strip()
        self.credits = credits
        self.teacher = teacher

    def to_dict(self) -> Mapping[str, object]:
        return {"id": self.id, "title": self.title, "credits": self.credits.ects, "teacher":
        self.teacher.name.full}

    def __str__(self) -> str:
        return f"{self.title} ({self.credits}, {self.teacher.name.initials})"
```

GradeBook — власна колекція і головний приклад інкапсуляції. Стан (_entries) приватний; єдині способи його змінити — enroll() і add(), і кожен перевіряє правило: двічі записатися не можна, поставити оцінку без запису не можна. Чотири dunder-методи вбудовують книжку в мову: len(book) — кількість оцінок, course in book — чи записаний, book[course] — оцінки з курсу, for course, grade in book — обхід усіх пар. earned_credits — property, що використовує Credits.__add__ через sum().

```
class GradeBook:
    """A student's record book: the courses he is enrolled in and the grades in each of them.

    A custom collection: len() counts grades, iteration yields (course, grade) pairs,
    `course in book` asks about enrollment, book[course] gives that course's grades.
    """

    __slots__ = ("_entries",)

    def __init__(self) -> None:
        self._entries: dict[Course, list[Grade]] = {}

    def enroll(self, course: Course) -> None:
        if course in self:
            raise AlreadyEnrolledError(f"курс «{course.title}» уже в заліковій книжці")
        self._entries[course] = []

    def add(self, course: Course, grade: Grade) -> None:
        if course not in self:
            raise NotEnrolledError(f"немає запису на курс «{course.title}», оцінку ставити нікуди")
        self._entries[course].append(grade)
```

```

@property
def courses(self) -> tuple[Course, ...]:
    return tuple(self._entries)

@property
def grades(self) -> tuple[Grade, ...]:
    return tuple(grade for grades in self._entries.values() for grade in grades)

@property
def average(self) -> float | None:
    grades = self.grades
    return mean(g.points for g in grades) if grades else None

@property
def earned_credits(self) -> Credits:
    """Credits of every course whose average is a pass; sum() works thanks to Credits.__add__."""
    passed = (
        course.credits
        for course, grades in self._entries.items()
        if grades and mean(g.points for g in grades) >= PASSING_THRESHOLD
    )
    return sum(passed, start=ZERO_CREDITS)

def average_for(self, course: Course) -> float | None:
    grades = self[course]
    return mean(g.points for g in grades) if grades else None

def __len__(self) -> int:
    return len(self.grades)

def __iter__(self) -> Iterator[tuple[Course, Grade]]:
    for course, grades in self._entries.items():
        for grade in grades:
            yield course, grade

def __contains__(self, item: object) -> bool:
    return item in self._entries

def __getitem__(self, course: Course) -> tuple[Grade, ...]:
    if course not in self:
        raise NotEnrolledError(f"немає запису на курс «{course.title}»")
    return tuple(self._entries[course])

def __repr__(self) -> str:
    return f"GradeBook(courses={len(self._entries)}, grades={len(self)})"

```

Student **має** GradeBook і делегує йому enroll() / record(); сам він зберігає лише групу за read-only property та e-mail. from_payload — class factory: класовий метод, який будує сутність із TypedDict (розділ 10); to_dict — зворотнє перетворення. average і contact — обчислювані properties, за якими ззовні не видно, що стоїть колекція.

```

class Student(Person):
    """A student of one group with his own GradeBook (composition: Student *has* a GradeBook)."""

```

```

__slots__ = ("_group", "_book", "email")

def __init__(
    self,
    entity_id: EntityId,
    name: PersonName,
    group: GroupCode,
    email: str | None = None,
    book: GradeBook | None = None,
) -> None:
    super().__init__(entity_id, name)
    self._group = group
    self._book = book if book is not None else GradeBook()
    self.email = email

@classmethod
def from_payload(cls, data: StudentPayload) -> Student:
    """Class factory: build an entity from an external dict shape (TypedDict)."""
    return cls(
        data["id"],
        PersonName(data["first_name"], data["last_name"]),
        GroupCode(data["group"]),
        data.get("email"),
    )

def to_dict(self) -> StudentPayload:
    """The inverse of from_payload; together they make Student structurally Serializable."""
    payload: StudentPayload = {
        "id": self.id,
        "first_name": self.name.first,
        "last_name": self.name.last,
        "group": str(self._group),
    }
    if self.email is not None:
        payload["email"] = self.email
    return payload

@property
def group(self) -> GroupCode:
    return self._group

@property
def book(self) -> GradeBook:
    return self._book

@property
def average(self) -> float | None:
    return self._book.average

@property
def contact(self) -> str:
    return self.email or self.name.full

def enroll(self, course: Course) -> None:

```

```

self._book.enroll(course)

def record(self, course: Course, grade: Grade) -> None:
    self._book.add(course, grade)

def __str__(self) -> str:
    return f"{super().__str__()}, {self._group}"

```

Group — сутність, яка одночасно є колекцією студентів. `add()` перевіряє, що код групи студента збігається з кодом групи, і відхиляє дублікат; `__contains__` приймає об'єкт, як вимагає контракт оператора `in`, і сам звужує тип через `isinstance`.

```

class Group(Entity):
    """Academic group: a code and the roster. Iterable, sized, supports `student in group`."""

    __slots__ = ("code", "_students")

    def __init__(self, entity_id: EntityId, code: GroupCode) -> None:
        super().__init__(entity_id)
        self.code = code
        self._students: dict[EntityId, Student] = {}

    def add(self, student: Student) -> None:
        if student.group != self.code:
            raise WrongGroupError(f"{student.name} належить до групи {student.group}, а не {self.code}")
        if student in self:
            raise DuplicateEntityError(f"{student.name} уже в групі {self.code}")
        self._students[student.id] = student

    @property
    def average(self) -> float | None:
        averages = [s.average for s in self if s.average is not None]
        return mean(averages) if averages else None

    def __len__(self) -> int:
        return len(self._students)

    def __iter__(self) -> Iterator[Student]:
        return iter(self._students.values())

    def __contains__(self, item: object) -> bool:
        return isinstance(item, Student) and item.id in self._students

    def __str__(self) -> str:
        return f"{self.code} ({len(self)} студ.)"

```

7. Ієрархія винятків

Кожна помилка моделі — нащадок `DomainError`, а той — нащадок `StudentManagerError` з ЛР1. Це дає три рівні перехоплення: конкретний виняток у тесті, `DomainError` у сервісі, `StudentManagerError` у меню чи майбутньому API. Винятки додатково успадковують

LookupError або ValueError, тож код, що не знає про проєкт, теж їх обробить. Перевірка балу та назви групи використовує InvalidGradeError / InvalidGroupError з ЛР1 без змін.

```
"""Exceptions of the domain layer.
```

```
Every class below descends from the project-wide StudentManagerError (lab 1), so the console menu
and future API handlers keep catching one base type. Grade and group validation reuse
InvalidGradeError / InvalidGroupError from lab 1 unchanged.
"""
```

```
from __future__ import annotations
```

```
from student_manager.exceptions import StudentManagerError
```

```
class DomainError(StudentManagerError):
    """Base for every rule violation inside the object model."""
```

```
class EntityNotFoundError(DomainError, LookupError):
    """A repository lookup by id found nothing."""

    def __init__(self, kind: str, entity_id: int) -> None:
        super().__init__(f"{kind} з id={entity_id} не знайдено")
        self.kind = kind
        self.entity_id = entity_id
```

```
class DuplicateEntityError(DomainError, ValueError):
    """Two different objects claim the same identity or the same seat."""
```

```
class EnrollmentError(DomainError, ValueError):
    """Base for everything that can go wrong with course enrollment."""
```

```
class AlreadyEnrolledError(EnrollmentError):
    """The student is enrolled in this course already."""
```

```
class NotEnrolledError(EnrollmentError):
    """A grade for a course the student never enrolled in."""
```

```
class WrongGroupError(EnrollmentError):
    """A student put into a group whose code differs from his own."""
```

8. Абстракції: ABC і Protocol

8.1. Protocol — структурні інтерфейси

Три протоколи, кожен з одним членом — це interface segregation: репозиторію від об'єкта потрібен лише `id`, експорту — лише `to_dict()`, сервісам — лише `send()`. Клас підходить, якщо має потрібний метод; `ConsoleNotifier` і `RecordingNotifier` ніде не згадують `Notifier`, а муру приймає їх там, де оголошено `notifier: Notifier`. Дві деталі, які довелося з'ясувати: `id` у `HasId` оголошено як `property`, бо для звичайного атрибута `id: int` муру вимагає, щоб він був записуваним, а в `Entity` він `read-only`; `to_dict()` повертає `Mapping[str, object]`, а не `dict`, бо `TypedDict` сумісний з `Mapping`, але не з `dict`. `runtime_checkable` вмикає `isinstance(obj, Serializable)` — у демонстрації `Student`, `Grade` і `Course` проходять перевірку, `Teacher` і `str` — ні.

```
"""Structural interfaces. Nothing here is inherited from – an object fits when its shape fits.
```

```
Three small protocols instead of one big one (interface segregation): the repository needs only
an id, the exporter needs only to_dict(), the services need only send().
"""
```

```
from __future__ import annotations
```

```
from collections.abc import Mapping
from typing import Protocol, runtime_checkable
```

```
class HasId(Protocol):
```

```
    """Anything a repository can store. Declared as a property so read-only ids satisfy it."""
```

```
    @property
```

```
    def id(self) -> int: ...
```

```
@runtime_checkable
```

```
class Serializable(Protocol):
```

```
    """Can turn itself into a plain mapping; runtime_checkable so isinstance() works on mixed input."""
```

```
    def to_dict(self) -> Mapping[str, object]: ...
```

```
class Notifier(Protocol):
```

```
    """Delivery channel for messages to a person. Console today, e-mail or a queue later."""
```

```
    def send(self, recipient: str, message: str) -> None: ...
```

8.2. ABC — номінальні контракти

ABC я використав там, де крім контракту потрібна спільна реалізація або де важливо, щоб клас усвідомлено належав до ієрархії. `Repository[T]` (розділ 9) оголошує чотири абстрактні примітиви, а `require()`, `find()`, `__contains__`, `__len__` реалізує поверх них — кожна майбутня реалізація отримує їх безкоштовно. `AcademicStatusPolicy` — стратегія з одним абстрактним

методом; екземпляр самої policy створити не можна, лише нащадка. StrictPolicy наслідує ThresholdPolicy і через super().status_of() лише додає одне правило поверх батьківського.

```
class AcademicStatusPolicy(ABC):
    """Strategy: how a student's grades turn into a status. Swappable without touching services."""

    @abstractmethod
    def status_of(self, student: Student) -> AcademicStatus: ...

class ThresholdPolicy(AcademicStatusPolicy):
    """Looks only at the average."""

    def __init__(self, excellent: float = EXCELLENT_THRESHOLD, passing: float = PASSING_THRESHOLD) -> None:
        self.excellent = excellent
        self.passing = passing

    def status_of(self, student: Student) -> AcademicStatus:
        average = student.average
        if average is None:
            return AcademicStatus.NO_GRADES
        if average >= self.excellent:
            return AcademicStatus.EXCELLENT
        if average >= self.passing:
            return AcademicStatus.GOOD
        return AcademicStatus.AT_RISK

class StrictPolicy(ThresholdPolicy):
    """Same thresholds, but a single failing grade puts the student at risk whatever his average."""

    def status_of(self, student: Student) -> AcademicStatus:
        if any(not grade.is_passing for _, grade in student.book):
            return AcademicStatus.AT_RISK
        return super().status_of(student)
```

Різниця в одному реченні: ABC перевіряє належність до ієрархії (nominal typing) і робить це в runtime при створенні об'єкта; Protocol перевіряє форму (structural typing) і робить це type checker до запуску.

9. Generic repository: TypeVar і Generic[T]

T = TypeVar("T", bound=HasId) — bound TypeVar: підставити можна будь-який тип, який структурно має id, а всередині класу до item.id можна звертатися безпечно. Repository(ABC, Generic[T]) — абстрактний контракт сховища; InMemoryRepository[T] — dict-реалізація, параметризована в місці виклику: InMemoryRepository[Student]() приймає лише Student, InMemoryRepository[Teacher]() — лише Teacher, і саме це ловить третя навмисна помилка в розділі 16. Сервіси типізовані абстракцією Repository[Student], тож реалізацію на SQLite чи PostgreSQL у наступних роботах можна буде підставити, не торкаючись жодного сервісу.

```
"""Storage abstraction and its in-memory implementation.
```

```
Repository[T] is the seam between the domain and infrastructure: services depend on the abstract class, the demo plugs in a dict-backed one, a database-backed one comes in a later lab without a single change in the services.
"""
```

```
from __future__ import annotations
```

```
from abc import ABC, abstractmethod
from collections.abc import Callable, Iterable, Iterator
from typing import Generic, TypeVar
```

```
from student_manager.domain.errors import DuplicateEntityError, EntityNotFoundError
from student_manager.domain.protocols import HasId
```

```
# Bound TypeVar: T may be any type, as long as it structurally has an `id`.
T = TypeVar("T", bound=HasId)
```

```
class Repository(ABC, Generic[T]):
    """Abstract contract. The concrete helpers (require, __contains__, __len__) are built on the
    abstract primitives, so every implementation gets them for free."""

    @abstractmethod
    def add(self, item: T) -> None: ...

    @abstractmethod
    def get(self, item_id: int) -> T | None: ...

    @abstractmethod
    def all(self) -> list[T]: ...

    @abstractmethod
    def remove(self, item_id: int) -> bool: ...

    def require(self, item_id: int) -> T:
        """get() that raises instead of returning None; keeps `if x is None` out of the services."""
        item = self.get(item_id)
        if item is None:
            raise EntityNotFoundError(self.kind, item_id)
        return item

    def find(self, predicate: Callable[[T], bool]) -> list[T]:
        return [item for item in self.all() if predicate(item)]

    @property
    def kind(self) -> str:
        """Human name of the stored type for error messages; overridable."""
        return "об'єкт"

    def __contains__(self, item_id: object) -> bool:
        return isinstance(item_id, int) and self.get(item_id) is not None
```

```

def __len__(self) -> int:
    return len(self.all())

class InMemoryRepository(Repository[T]):
    """dict-backed repository, parameterised at the call site: InMemoryRepository[Student]()"""

    def __init__(self, items: Iterable[T] = (), kind: str = "об'єкт") -> None:
        self._items: dict[int, T] = {}
        self._kind = kind
        for item in items:
            self.add(item)

    def add(self, item: T) -> None:
        if item.id in self._items:
            raise DuplicateEntityError(f"{self._kind} з id={item.id} уже існує")
        self._items[item.id] = item

    def get(self, item_id: int) -> T | None:
        return self._items.get(item_id)

    def all(self) -> list[T]:
        return list(self._items.values())

    def remove(self, item_id: int) -> bool:
        return self._items.pop(item_id, None) is not None

    @property
    def kind(self) -> str:
        return self._kind

    def __iter__(self) -> Iterator[T]:
        return iter(self._items.values())

    def __repr__(self) -> str:
        return f"InMemoryRepository[{self._kind}]({len(self)} items)"

```

10. TypedDict і DTO

TypedDict описує форму словника для type checker, але в runtime лишається звичайним dict: без валідації, методів та ідентичності. Тому він живе тільки на межі системи. StudentPayload — те, що прийде з JSON або CSV, з NotRequired[str] для e-mail, якого може не бути; GradePayload — рядок імпорту оцінок; StudentReport і GroupReport — те, що ReportingService віддає назовні. Перетворення payload → сутність відбувається в одному місці — Student.from_payload() — і саме там рядок групи стає валідованим GroupCode, а сирі рядки імен — PersonName. Зворотний шлях — Student.to_dict(), і туру стежить, що словник має рівно ті ключі й типи, які оголошено.

```

"""Dict shapes for the outside world (JSON, CSV rows, a future REST API) and converters.

```

A TypedDict is still a dict at runtime: no validation, no methods, no identity. That is exactly why

it stays at the boundary: Student.from_payload() / Student.to_dict() and group_to_report() are the only places where a payload becomes an entity or an entity becomes a payload.

```
"""
```

```
from __future__ import annotations
```

```
from typing import NotRequired, TypedDict
```

```
from student_manager.domain.models import Group
from student_manager.domain.value_objects import AcademicStatus
```

```
class StudentPayload(TypedDict):
    """Incoming student record; email may be absent."""

    id: int
    first_name: str
    last_name: str
    group: str
    email: NotRequired[str]
```

```
class GradePayload(TypedDict):
    """One row of a grade import: who, which course, how many points."""

    student_id: int
    course_id: int
    points: float
```

```
class StudentReport(TypedDict):
    """What the reporting service says about one student."""

    id: int
    name: str
    group: str
    courses: int
    grades: int
    average: float | None
    credits: int
    status: AcademicStatus
```

```
class GroupReport(TypedDict):
    code: str
    students: int
    average: float | None
    best: str | None
```

```
def group_to_report(group: Group) -> GroupReport:
    graded = [s for s in group if s.average is not None]
    best = max(graded, key=lambda s: s.average or 0.0, default=None)
```

```

return {
    "code": str(group.code),
    "students": len(group),
    "average": group.average,
    "best": best.name.full if best is not None else None,
}

@classmethod
def from_payload(cls, data: StudentPayload) -> Student:
    """Class factory: build an entity from an external dict shape (TypedDict)."""
    return cls(
        data["id"],
        PersonName(data["first_name"], data["last_name"]),
        GroupCode(data["group"]),
        data.get("email"),
    )

def to_dict(self) -> StudentPayload:
    """The inverse of from_payload; together they make Student structurally Serializable."""
    payload: StudentPayload = {
        "id": self.id,
        "first_name": self.name.first,
        "last_name": self.name.last,
        "group": str(self._group),
    }
    if self.email is not None:
        payload["email"] = self.email
    return payload

```

11. Application service layer i dependency injection

Сервіси не містять правил предметної області — правила сидять у сутностях. Сервіс дістає потрібні об'єкти з репозиторіїв, викликає метод сутності й повідомляє студента. Три сервіси — три use case: зарахування і запис на курси, виставлення оцінок зі статусом, звіти. Кожна залежність приходить через `__init__` і типізована абстракцією (Repository, Notifier, AcademicStatusPolicy), а не конкретним класом — це і є dependency injection.

```

class EnrollmentService:
    """Admits students to groups and enrolls them in courses."""

    def __init__(
        self,
        students: Repository[Student],
        courses: Repository[Course],
        groups: Repository[Group],
        notifier: Notifier,
    ) -> None:
        self._students = students
        self._courses = courses
        self._groups = groups
        self._notifier = notifier

    def admit(self, student: Student) -> Group:
        """Register a new student and seat him in the group his code points to."""
        group = next(iter(self._groups.find(lambda g: g.code == student.group)), None)

```

```

        if group is None:
            group = Group(len(self._groups) + 1, student.group)
            self._groups.add(group)
            self._students.add(student)
            group.add(student)
            self._notifier.send(student.contact, f"Вас зараховано до групи {group.code}")
            return group

    def enroll(self, student_id: int, course_id: int) -> Student:
        student = self._students.require(student_id)
        course = self._courses.require(course_id)
        student.enroll(course)
        self._notifier.send(student.contact, f"Вас записано на курс «{course.title}»")
        return student

class GradingService:
    """Records grades and answers the two questions everybody asks: average and status."""

    def __init__(
        self,
        students: Repository[Student],
        courses: Repository[Course],
        notifier: Notifier,
        policy: AcademicStatusPolicy,
    ) -> None:
        self._students = students
        self._courses = courses
        self._notifier = notifier
        self._policy = policy

    def grade(self, student_id: int, course_id: int, points: float) -> Grade:
        student = self._students.require(student_id)
        course = self._courses.require(course_id)
        grade = Grade(points)
        student.record(course, grade)
        if not grade.is_passing:
            self._notifier.send(student.contact, f"Незадовільна оцінка з «{course.title}»: {grade}")
        return grade

    def average_of(self, student_id: int) -> float | None:
        return self._students.require(student_id).average

    def status_of(self, student_id: int) -> AcademicStatus:
        return self._policy.status_of(self._students.require(student_id))

class ReportingService:
    """Read-only views over the repositories: rankings, per-student and per-group reports."""

    def __init__(self, students: Repository[Student], groups: Repository[Group], policy:
AcademicStatusPolicy) -> None:
        self._students = students
        self._groups = groups
        self._policy = policy

```

```

def ranking(self) -> list[tuple[Student, float]]:
    graded = [(s, s.average) for s in self._students.all() if s.average is not None]
    return sorted(graded, key=lambda pair: (-pair[1], pair[0].name.full))

def student_report(self, student_id: int) -> StudentReport:
    student = self._students.require(student_id)
    return {
        "id": student.id,
        "name": student.name.full,
        "group": str(student.group),
        "courses": len(student.book.courses),
        "grades": len(student.book),
        "average": student.average,
        "credits": student.book.earned_credits.ects,
        "status": self._policy.status_of(student),
    }

def group_reports(self) -> list[GroupReport]:
    return [group_to_report(g) for g in sorted(self._groups.all(), key=lambda g: str(g.code))]

def with_status(self, status: AcademicStatus) -> list[Student]:
    return self._students.find(lambda s: self._policy.status_of(s) is status)

@staticmethod
def export(objects: Iterable[object]) -> list[Mapping[str, object]]:
    """Serialise whatever knows how; the runtime_checkable protocol filters the rest out."""
    return [obj.to_dict() for obj in objects if isinstance(obj, Serializable)]

```

11.1. Реалізації Notifier

Дві реалізації одного протоколу: консольна для демонстрації та журнальна для тестів. Жодна не успадковує Notifier — достатньо методу `send()` з тією самою сигнатурою.

```

"""Concrete notifiers. Neither inherits from the Notifier protocol – they simply have send()."""

```

```

from __future__ import annotations

```

```

class ConsoleNotifier:
    """Prints the message; what the demo uses."""

    def send(self, recipient: str, message: str) -> None:
        print(f" [повідомлення → {recipient}] {message}")

```

```

class RecordingNotifier:
    """Keeps every message in a list instead of delivering it; what the tests inject."""

    def __init__(self) -> None:
        self.sent: list[tuple[str, str]] = []

    def send(self, recipient: str, message: str) -> None:
        self.sent.append((recipient, message))

```

```
def __len__(self) -> int:
    return len(self.sent)
```

11.2. Ручний DI-контейнер

`Container.build()` — composition root: єдине місце в пакеті, де абстракції зустрічаються з конкретними класами. Демонстрація викликає `Container.build(notifier=RecordingNotifier())`, тести — те саме, а щоб перейти на консольні повідомлення й сувору policy, достатньо змінити два аргументи. Сам контейнер — frozen dataclass, бо після збирання його не повинні змінювати.

```
"""Hand-written dependency injection container: the one place where concrete classes meet.
```

```
Nothing else in the package knows that repositories are dicts or that notifications go to the
console. Swap a field here (or pass another object to build()) and every service follows.
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from student_manager.domain.models import Course, Group, Student
from student_manager.domain.notifications import ConsoleNotifier
from student_manager.domain.protocols import Notifier
from student_manager.domain.repositories import InMemoryRepository, Repository
from student_manager.domain.services import (
    AcademicStatusPolicy,
    EnrollmentService,
    GradingService,
    ReportingService,
    ThresholdPolicy,
)
```

```
@dataclass(frozen=True, slots=True)
```

```
class Container:
```

```
    students: Repository[Student]
    courses: Repository[Course]
    groups: Repository[Group]
    notifier: Notifier
    policy: AcademicStatusPolicy
    enrollment: EnrollmentService
    grading: GradingService
    reporting: ReportingService
```

```
    @classmethod
```

```
    def build(cls, notifier: Notifier | None = None, policy: AcademicStatusPolicy | None = None) ->
```

```
Container:
```

```
    """Composition root. Defaults are the demo wiring; tests pass their own doubles."""
    notifier = notifier if notifier is not None else ConsoleNotifier()
    policy = policy if policy is not None else ThresholdPolicy()
    students: Repository[Student] = InMemoryRepository(kind="студент")
    courses: Repository[Course] = InMemoryRepository(kind="курс")
    groups: Repository[Group] = InMemoryRepository(kind="група")
```

```

return cls(
    students=students,
    courses=courses,
    groups=groups,
    notifier=notifier,
    policy=policy,
    enrollment=EnrollmentService(students, courses, groups, notifier),
    grading=GradingService(students, courses, notifier, policy),
    reporting=ReportingService(students, groups, policy),
)

```

12. Принципи SOLID у коді

S — Single Responsibility. GradeBook відповідає лише за оцінки студента, Group — лише за склад групи, ConsoleNotifier — лише за доставку, ReportingService — лише за читання й агрегацію. Коли потрібно змінити формат звіту, я відкриваю один файл, і він не має жодного стосунку до того, як ставляться оцінки.

O — Open/Closed. Нова academic policy — новий клас-нащадок AcademicStatusPolicy; новий канал повідомлень — новий клас із методом send(); новий формат рядка імпорту — новий RowFormatter. StrictPolicy і TableFormatter я додав уже після того, як сервіси й GradeImporter були написані, і не змінив у них жодного рядка.

L — Liskov Substitution. StrictPolicy підставляється замість ThresholdPolicy: та сама сигнатура, той самий тип результату, лише суворіше правило — GradingService не помічає різниці. InMemoryRepository підставляється замість Repository. У протилежний бік: Entity.__eq__ повертає NotImplemented для чужих типів, а не False, щоб не ламати контракт порівняння в Python.

I — Interface Segregation. Замість одного «інтерфейсу сутності» — три протоколи по одному методу (HasId, Serializable, Notifier) і три для імпорту (LineParser, GradeValidator, RowFormatter). RecordingNotifier реалізує тільки send(), і йому не доводиться писати заглушки для того, чого сервіси не викликають.

D — Dependency Inversion. GradingService залежить від Repository[Student] і Notifier — абстракцій, які він сам і визначає своїм контрактом, — а не від dict і print. Напрямок залежності розвернуто: не сервіс знає про сховище, а сховище підлаштовується під контракт сервісу. Конкретні класи з'являються лише в Container.build().

13. Демонстраційний сценарій

__main__.py наповнює модель через сервіси, а не в обхід них: seed() створює викладачів і курси, зараховує дев'ятьох студентів через admit(), записує кожного на чотири курси й ставить 36 оцінок із даних ЛР2. Далі кожен розділ виводу ілюструє одну групу вимог. Фрагмент із наповненням і підміною залежностей:

```

def seed(app: Container) -> None:
    """Fill the repositories through the services, never behind their back."""

```

```

for teacher_id, first, last, department in TEACHERS: # teachers live inside their courses
    teacher = Teacher(teacher_id, PersonName(first, last), department)
    for course_id, title, credits, owner in COURSES:
        if owner == teacher_id:
            app.courses.add(Course(course_id, title, Credits(credits), teacher))
for payload in STUDENTS:
    student = Student.from_payload(payload)
    app.enrollment.admit(student)
    for (course_id, _, _, _), points in zip(COURSES, GRADES[student.id], strict=True):
        app.enrollment.enroll(student.id, course_id)
        app.grading.grade(student.id, course_id, points)

show("STRATEGY: ThresholdPolicy проти StrictPolicy після одного «незадовільно»")
app.grading.grade(7, 11, 55) # Скиба Юрій, Мова С, друга оцінка
strict = ReportingService(app.students, app.groups, StrictPolicy())
for student_id in (7, 8):
    s = app.students.require(student_id)
    print(f" {s.name.full:<16} avg {s.average:5.2f} threshold:
{app.grading.status_of(student_id):<18} "
          f"strict: {strict.student_report(student_id)['status']}")
    print(f" останнє повідомлення: {log.sent[-1]}")

show("TYPEDDICT <-> ENTITY, runtime_checkable Protocol")
payload: StudentPayload = {"id": 10, "first_name": "Оксана", "last_name": "Ференц", "group": "ФЕП-
31с"}
newcomer = Student.from_payload(payload)
print(f" from_payload -> {newcomer}; to_dict -> {newcomer.to_dict()}")
print(f" to_dict з email -> {app.students.require(1).to_dict()}")
for thing in (newcomer, Grade(77), python_course, teacher, "рядок"):
    print(f" isinstance({type(thing).__name__}, Serializable) = {isinstance(thing,
Serializable)}")
print(" export:", ReportingService.export([Grade(77), teacher, python_course]))

show("DEPENDENCY INJECTION: той самий сервіс, інший Notifier і інша policy")
console_app = Container.build(notifier=ConsoleNotifier(), policy=StrictPolicy())
console_app.courses.add(Course(11, "Мова С", Credits(4), teacher))
console_app.enrollment.admit(newcomer)
console_app.enrollment.enroll(10, 11)
console_app.grading.grade(10, 11, 42)
print(f" статус: {console_app.grading.status_of(10)}")

```

14. Результат роботи програми

Повний вивід `python -m student_manager.domain`. Замість скриншота — текст консолі, як і в попередніх звітах; він ідентичний тому, що видно у вікні терміналу.

```

VALUE OBJECTS: frozen dataclass, properties, total_ordering, __add__
-----
Grade(95) -> 95.0 (A); Grade(58) -> 58.0 (FX); is_passing: True, False
Grade(95) > Grade(58): True; Grade(90) == Grade(90.0): True
sorted: ['60.5 (E)', '81.0 (B)', '95.0 (A)']
max: 95.0 (A)
best.points = 100 -> FrozenInstanceError: cannot assign to field 'points'
Grade(101): InvalidGradeError: середній бал має бути в межах 0..100, отримано 101
Credits(4) + Credits(4) + Credits(3) = 11 кред.

```

```

PersonName -> PersonName(first='Марта', last='Гнатишин'); full='Гнатишин Марта'; initials='Гнатишин
М.'
GroupCode -> faculty=ФЕП, stream=31
GroupCode("група 31"): InvalidGroupError: назва групи 'група 31' не схожа на ФЕП-31с

ENTITIES: identity, inheritance, super()
-----
Student(id=1, name='Гнатишин Марта') == Student(id=1, name='Гнатишин-Ковальчук Марта'): True (одна
id -> одна сутність)
Student(id=1) == Teacher(id=1): False (різні типи)
len({marta, same_id}) = 1
str(student): Гнатишин Марта, ФЕП-31с; str(teacher): Ткаченко Олена (кафедра програмування)
MRO Student: Student -> Person -> Entity -> object

СЕРВІСИ ЧЕРЕЗ DI-КОНТЕЙНЕР: admit, enroll, grade
-----
студентів: 9, курсів: 4, груп: 3
повідомлень у RecordingNotifier: 45; перші два:
marta.h@lnu.edu.ua: Вас зараховано до групи ФЕП-31с
marta.h@lnu.edu.ua: Вас записано на курс «Професійний Python»

GRADEBOOK – власна колекція: __len__, __contains__, __getitem__, __iter__
-----
GradeBook(courses=4, grades=4); len(book) = 4
python_course in book: True; book[python_course] = ['95.0 (A)']
Професійний Python 95.0 (A)
Мова С 91.0 (A)
Методи обчислень 94.0 (A)
Інженерія даних 93.0 (A)
average = 93.25; earned_credits = 14 кред.
record без enroll: NotEnrolledError: немає запису на курс «Фізика», оцінку ставити нікуди
enroll вдруге: AlreadyEnrolledError: курс «Професійний Python» уже в заліковій книжці

GROUP – сутність-колекція
-----
ФЕП-31с (3 студ.): Гнатишин М., Дзюба О., Вовк Т.; середній 85.75
ФЕП-32 (3 студ.): Кравець С., Пасічник І., Мельник А.; середній 86.00
ФЕП-33 (3 студ.): Скиба Ю., Ковальчук Н., Тимків Б.; середній 81.42
student in fer31: True
add ФЕП-32 у ФЕП-31с: WrongGroupError: Кравець Соломія належить до групи ФЕП-32, а не ФЕП-31с

REPORTING: рейтинг, статуси (ThresholdPolicy), звіти-TypedDict
-----
1. Пасічник Ірина ФЕП-32 96.50
2. Гнатишин Марта ФЕП-31с 93.25
3. Тимків Богдан ФЕП-33 90.50
4. Кравець Соломія ФЕП-32 88.00
5. Вовк Тарас ФЕП-31с 85.00
6. Ковальчук Назар ФЕП-33 82.50
7. Дзюба Остап ФЕП-31с 79.00
8. Мельник Андрій ФЕП-32 73.50
9. Скиба Юрій ФЕП-33 71.25
відмінники: ['Гнатишин Марта', 'Пасічник Ірина', 'Тимків Богдан']
student_report(9): {'id': 9, 'name': 'Тимків Богдан', 'group': 'ФЕП-33', 'courses': 4, 'grades': 4,
'average': 90.5, 'credits': 14, 'status': <AcademicStatus.EXCELLENT: 'відмінник'>}
ФЕП-31с студентів 3 середній 85.75 кращий: Гнатишин Марта
ФЕП-32 студентів 3 середній 86.00 кращий: Пасічник Ірина

```

ФЕП-33 студентів 3 середній 81.42 кращий: Тимків Богдан

STRATEGY: ThresholdPolicy проти StrictPolicy після одного «незадовільно»

Скиба Юрій avg 68.00 threshold: встигає strict: ризик відрахування
Ковальчук Назар avg 82.50 threshold: встигає strict: встигає
останнє повідомлення: ('Скиба Юрій', 'Незадовільна оцінка з «Мова С»: 55.0 (FX)')

TYPEDDICT <-> ENTITY, runtime_checkable Protocol

from_payload -> Ференц Оксана, ФЕП-31с; to_dict -> {'id': 10, 'first_name': 'Оксана', 'last_name': 'Ференц', 'group': 'ФЕП-31с'}
to_dict з email -> {'id': 1, 'first_name': 'Марта', 'last_name': 'Гнатишин', 'group': 'ФЕП-31с', 'email': 'marta.h@lnu.edu.ua'}
isinstance(Student, Serializable) = True
isinstance(Grade, Serializable) = True
isinstance(Course, Serializable) = True
isinstance(Teacher, Serializable) = False
isinstance(str, Serializable) = False
export: [{'points': 77.0, 'ects': 'C'}, {'id': 10, 'title': 'Професійний Python', 'credits': 4, 'teacher': 'Ткаченко Олена'}]

DEPENDENCY INJECTION: той самий сервіс, інший Notifier і інша policy

[повідомлення -> Ференц Оксана] Вас зараховано до групи ФЕП-31с
[повідомлення -> Ференц Оксана] Вас записано на курс «Мова С»
[повідомлення -> Ференц Оксана] Незадовільна оцінка з «Мова С»: 42.0 (FX)
статус: ризик відрахування

GENERIC REPOSITORY: InMemoryRepository[Teacher]

InMemoryRepository[викладач](4 items); 2 in teachers: True; 9 in teachers: False
find(кафедра програмування): ['Ткаченко Олена (кафедра програмування)', 'Романюк Ігор (кафедра системного програмування)']
remove(4): True; remove(4) вдруге: False; len: 3
require(4): EntityNotFoundError: викладач з id=4 не знайдено
add дублікату id=1: DuplicateEntityError: викладач з id=1 уже існує

ІЄРАРХІЯ ВИНЯТКІВ: усе ловиться одним DomainError / StudentManagerError

except DomainError: EntityNotFoundError – студент з id=99 не знайдено
except StudentManagerError: EntityNotFoundError – студент з id=99 не знайдено

ЕКСПЕРИМЕНТ: inheritance (CsvGradeImporter) проти composition (GradeImporter)

однаковий результат: True
1:10 -> 95.0 (A)
7:11 -> 55.0 (FX)
9:13 -> 92.0 (A)
інший формат і роздільник без нового підкласу:
1	10	95.0	A
7	11	55.0	FX
9	13	92.0	A

15. Експериментальна частина

15.1. Сервіс створює залежність сам (варіант А) проти dependency injection (варіант В)

Варіант А у термінах проєкту виглядав би так (у коді його немає — саме від нього я відмовився):

```
class GradingService:                                # варіант А
    def __init__(self, students: Repository[Student], courses: Repository[Course]) -> None:
        self._students = students
        self._courses = courses
        self._notifier = ConsoleNotifier()           # сервіс сам обирає реалізацію
        self._policy = ThresholdPolicy()              # і сам обирає правило
```

Варіант В — те, що реалізовано: `GradingService(students, courses, notifier, policy)`. Тест нижче можливий лише у варіанті В: він підставляє `RecordingNotifier` і перевіряє, що після незадовільної оцінки сервіс надіслав саме те повідомлення, — без патчів, без перехоплення `stdout`.

```
def test_enroll_grade_average_and_status(app: tuple[Container, RecordingNotifier]) -> None:
    container, log = app
    container.enrollment.admit(student(1))
    container.enrollment.enroll(1, 10)
    container.enrollment.enroll(1, 11)
    assert container.grading.status_of(1) is AcademicStatus.NO_GRADES
    container.grading.grade(1, 10, 95)
    container.grading.grade(1, 11, 91)
    assert container.grading.average_of(1) == pytest.approx(93)
    assert container.grading.status_of(1) is AcademicStatus.EXCELLENT
    container.grading.grade(1, 11, 40)
    assert log.sent[-1] == ("Гнатишин Марта", "Незадовільна оцінка з «Мова С»: 40.0 (FX)")
    container.courses.add(course(12, "Методи обчислень", 3))
    with pytest.raises(NotEnrolledError):
        container.grading.grade(1, 12, 70)
    with pytest.raises(EntityNotFoundError):
        container.grading.grade(1, 99, 70)
```

Критерій	А: залежність усередині	В: dependency injection
Тестування	треба патчити <code>ConsoleNotifier</code> або ловити <code>stdout</code> ; тест знає про внутрішню будову	передаємо <code>RecordingNotifier</code> у конструктор; тест перевіряє лише поведінку
Заміна реалізації	правка коду сервісу (і повторне тестування сервісу)	інший аргумент у <code>Container.build()</code> ; сервіс не змінюється
Відповідність DIP	порушено: high-level сервіс імпортує low-level клас	дотримано: сервіс знає лише <code>Protocol Notifier</code> і <code>ABC policy</code>
Coupling	жорсткий: імпорт класу, знання про його конструктор	слабкий: лише сигнатура <code>send()</code> / <code>status_of()</code>

Прозорість	залежності приховані в тілі; щоб їх побачити, треба читати код	залежності перелічені в сигнатурі __init__
------------	--	---

Composition з abstraction знижує coupling, бо сервіс тримає посилання не на клас, а на контракт: усе, що він знає про notifier, — це один метод. Скільки б реалізацій не з'явилося, кількість речей, які сервісу треба знати, не росте. У варіанті А кожна нова реалізація — це ще один імпорт і ще одна гілка if усередині сервісу.

15.2. Inheritance проти composition на одному завданні

Завдання: перетворити рядки «student_id,course_id,points» на перевірені й відформатовані записи. Варіант А — базовий клас із template method: import_lines() фіксує алгоритм parse → validate → format, нащадок CsvGradeImporter доповнює лише розбір рядка. Варіант В — GradeImporter, який *має* parser, validator і formatter за трьома малими Protocol.

```
"""The same behaviour built two ways – the inheritance-vs-composition experiment of the lab.
```

```
Task: turn text lines "student_id,course_id,points" into validated, formatted rows.
```

```
Variant A: a base class with a template method; a subclass fills in the parsing step.
```

```
Variant B: a plain class that *has* a parser, a validator and a formatter, each behind its own small Protocol.
```

```
"""
```

```
from __future__ import annotations
```

```
from abc import ABC, abstractmethod
from collections.abc import Iterable
from typing import Protocol
```

```
from student_manager.domain.dto import GradePayload
from student_manager.domain.value_objects import Grade
```

```
# ----- variant A: inheritance
```

```
class BaseGradeImporter(ABC):
```

```
    """Template method: import_lines() fixes the algorithm, subclasses may only vary the steps."""
```

```
    def import_lines(self, lines: Iterable[str]) -> list[str]:
        return [self.format_row(self.validate(self.parse_line(line))) for line in lines if
line.strip()]
```

```
    @abstractmethod
```

```
    def parse_line(self, line: str) -> GradePayload: ...
```

```
    def validate(self, row: GradePayload) -> tuple[GradePayload, Grade]:
        return row, Grade(row["points"])
```

```
    def format_row(self, checked: tuple[GradePayload, Grade]) -> str:
```

```

        row, grade = checked
        return f"{row['student_id']}:{row['course_id']} -> {grade}"

class CsvGradeImporter(BaseGradeImporter):
    def parse_line(self, line: str) -> GradePayload:
        student_id, course_id, points = (part.strip() for part in line.split(","))
        return {"student_id": int(student_id), "course_id": int(course_id), "points": float(points)}

# ----- variant B: composition

class LineParser(Protocol):
    def parse(self, line: str) -> GradePayload: ...

class GradeValidator(Protocol):
    def validate(self, row: GradePayload) -> Grade: ...

class RowFormatter(Protocol):
    def format(self, row: GradePayload, grade: Grade) -> str: ...

class DelimitedParser:
    def __init__(self, delimiter: str = ",") -> None:
        self.delimiter = delimiter

    def parse(self, line: str) -> GradePayload:
        student_id, course_id, points = (part.strip() for part in line.split(self.delimiter))
        return {"student_id": int(student_id), "course_id": int(course_id), "points": float(points)}

class RangeValidator:
    def validate(self, row: GradePayload) -> Grade:
        return Grade(row["points"])

class ArrowFormatter:
    def format(self, row: GradePayload, grade: Grade) -> str:
        return f"{row['student_id']}:{row['course_id']} -> {grade}"

class TableFormatter:
    def format(self, row: GradePayload, grade: Grade) -> str:
        return f"| {row['student_id']:>3} | {row['course_id']:>3} | {grade.points:5.1f} | {grade.ects:<2} |"

class GradeImporter:

```

```

"""Has-a parser, validator and formatter; knows nothing about how any of them works."""

def __init__(self, parser: LineParser, validator: GradeValidator, formatter: RowFormatter) -> None:
    self._parser = parser
    self._validator = validator
    self._formatter = formatter

def import_lines(self, lines: Iterable[str]) -> list[str]:
    result: list[str] = []
    for line in lines:
        if not line.strip():
            continue
        row = self._parser.parse(line)
        result.append(self._formatter.format(row, self._validator.validate(row)))
    return result

```

Обидва варіанти дають однаковий результат на однакових даних (розділ 14, останній блок). Різниця проявляється, коли щось треба змінити: у демонстрації інший роздільник і табличний формат отримано двома аргументами `GradeImporter(DelimitedParser(";"), RangeValidator(), TableFormatter())`. У варіанті А для того самого довелося б або писати новий підклас під кожну комбінацію (роздільник $\times$ формат), або перевизначати `format_row` — і кожна така зміна проходить через ієрархію.

Критерій	Inheritance (BaseGradeImporter)	Composition (GradeImporter)
Coupling	нащадок прив'язаний до базового класу: знає його методи, порядок викликів, сигнатури	importer знає лише три протоколи по одному методу; частини не знають одна про одну
Extensibility	нова варіація = новий підклас; комбінації варіацій множаться	нова варіація = новий маленький клас; комбінації збираються аргументами
Testability	крок можна перевірити лише через <code>import_lines()</code> усього класу	<code>DelimitedParser</code> , <code>RangeValidator</code> , <code>TableFormatter</code> тестуються окремо (<code>test_composed_parts_are_testable_alone</code>)
Повторне використання	<code>parse_line</code> існує тільки всередині ієрархії імпортера	<code>DelimitedParser</code> придатний для будь-якого іншого імпорту рядків
Обсяг коду	коротше: два класи	довше: три протоколи, чотири компоненти, один збирач

Чесний висновок: варіант А не поганий сам по собі — `template method` доречний, коли алгоритм справді фіксований, а варіюється один крок. Але щойно з'являється друга вісь варіації (у нас — формат виводу), наслідування починає множити класи, а композиція — ні. Тому в

основній моделі я залишив наслідування лише для Person і для пари policy, де воно виражає is-a, а все інше зібрано з частин.

16. Статична перевірка типів

16.1. Інструмент і конфігурація

Використано муру 1.18 (Pyright не ставив: муру уже був у проєкті з LP1 і працює однією командою без IDE). У `pyproject.toml` додано групу `dev = ["муру>=1.10"]` та секцію `[tool.муру]`, тому `strict`-режим увімкнено для всього проєкту, а не лише прапорцем у командному рядку. `python_version = "3.11"` збігається з `requires-python`. CI-подібна команда з README:

```
python -m pip install -e ".[test,dev]"
python -m муру src --strict && python -m pytest -q
```

```
[project.optional-dependencies]
test = ["pytest>=8.0"]
dev = ["муру>=1.10"]
```

```
[tool.муру]
python_version = "3.11"
strict = true
warn_unused_ignores = true
warn_return_any = true
```

Результат перевірки всього пакета після завершення роботи:

```
$ python -m муру src --strict
Success: no issues found in 26 source files
```

Дві помилки, які муру знайшов у моєму власному коді під час роботи і які довелося виправити: (1) у `HasId` атрибут `id`: `int` муру трактує як записуваний, і `read-only property` в `Entity` йому не відповідала — вирішено оголошенням `id` як `property` в самому протоколі; (2) у демонстрації я повторно використав ім'я `error` для змінної циклу після блоку `except ... as error` — муру повідомив «Trying to read deleted variable», бо Python видаляє змінну винятку наприкінці `except`; змінну перейменовано.

16.2. Експеримент: навмисні помилки типів

Файл `lab4/experiment/type_errors_before.py` лежить поза пакетом (щоб не псувати перевірку проєкту) і містить три обов'язкові помилки з методичних вказівок, адаптовані до моєї моделі, приклад із п. 66 та ще одну, характерну саме для `value objects`:

```
"""Навмисні помилки типів для експерименту з муру (LP4, п. 66 і 71). Файл НЕ входить до пакета.
```

```
Кожна з чотирьох конструкцій виконується інтерпретатором без жодної скарги – Python не перевіряє анотації під час виконання. Ловить їх лише статичний аналізатор.
"""
```

```
from student_manager.domain.models import Course, Student, Teacher
from student_manager.domain.repositories import InMemoryRepository
from student_manager.domain.value_objects import Credits, GroupCode, PersonName
```

```
# Error 1: змінна оголошена як int, а присвоєно рядок
student_id: int = "1"

# Error 2: функція обіцяє float, а повертає str
def average() -> float:
    return "90.5"

# Error 3: репозиторій, параметризований Student, отримує Teacher
students: InMemoryRepository[Student] = InMemoryRepository()
students.add(Teacher(7, PersonName("Олена", "Ткаченко"), "кафедра програмування"))

# Error 4 (як у п. 66 методички): id сутності очікує int, передано str
lecturer = Teacher(1, PersonName("Ігор", "Романюк"), "кафедра системного програмування")
course = Course("one", "Мова C", Credits(4), lecturer)

# Бонус: сирий рядок замість value object – конструктор Student вимагає GroupCode
newcomer = Student(10, PersonName("Оксана", "Ференц"), "ФЕП-31с")
print(student_id, average(), len(students), course, newcomer, GroupCode)
```

Діагностика туру — п'ять помилок, кожна з точним рядком, очікуваним і фактичним типом:

```
$ MYPYPATH=src python -m mypy --strict ../lab4/experiment/type_errors_before.py
..\lab4\experiment\type_errors_before.py:12: error: Incompatible types in assignment (expression has
type "str", variable has type "int") [assignment]
..\lab4\experiment\type_errors_before.py:17: error: Incompatible return value type (got "str", expected
"float") [return-value]
..\lab4\experiment\type_errors_before.py:22: error: Argument 1 to "add" of "InMemoryRepository" has
incompatible type "Teacher"; expected "Student" [arg-type]
..\lab4\experiment\type_errors_before.py:26: error: Argument 1 to "Course" has incompatible type "str";
expected "int" [arg-type]
..\lab4\experiment\type_errors_before.py:29: error: Argument 3 to "Student" has incompatible type
"str"; expected "GroupCode" [arg-type]
Found 5 errors in 1 file (checked 1 source file)
```

Показовий момент — що робить із цим самим файлом інтерпретатор. Перші три помилки виконуються мовчки: `student_id` спокійно стає рядком, `average()` повертає рядок, `Teacher` лягає в репозиторій студентів. Падає лише четверта, і то не тому, що Python перевіряв тип, а тому, що `Entity.__init__` порівнює `id` з нулем, а `"one" <= 0` для рядка не визначено. П'ята помилка (рядок замість `GroupCode`) теж пройшла б непоміченою до першого звернення до `group.faculty`.

```
$ python ../lab4/experiment/type_errors_before.py
Traceback (most recent call last):
  File "C:\Users\user\vscode\uni\python\lab4\experiment\type_errors_before.py", line 26, in <module>
    course = Course("one", "Мова C", Credits(4), lecturer)
    ~~~~~^~~~~~
  File "C:\Users\user\vscode\uni\python\project-lab4\src\student_manager\domain\models.py", line 97, in
__init__
    super().__init__(entity_id)
    ~~~~~^~~~~~
  File "C:\Users\user\vscode\uni\python\project-lab4\src\student_manager\domain\models.py", line 42, in
```

```

__init__
    if entity_id <= 0:
        ^^^^^^^^^^^^^^^^^
TypeError: '<=' not supported between instances of 'str' and 'int'

```

16.3. Виправлення та повторна перевірка

Виправлення: значення приведено до оголошених типів, викладач отримав власний `InMemoryRepository[Teacher]`, `id` став цілим, назву групи загорнуто в `GroupCode` — і саме останнє показує, навіщо `value objects`: помилка типу «рядок замість `GroupCode`» одночасно означає, що назва групи не пройшла валідацію.

```

"""Той самий сценарій після виправлення всіх діагностик мруу."""

from student_manager.domain.models import Course, Student, Teacher
from student_manager.domain.repositories import InMemoryRepository
from student_manager.domain.value_objects import Credits, GroupCode, PersonName

# Fix 1: значення відповідає оголошеному типу
student_id: int = 1

# Fix 2: повертаємо число, а не його текстове подання
def average() -> float:
    return 90.5

# Fix 3: у репозиторій студентів кладемо студента; викладачі мають власний репозиторій
students: InMemoryRepository[Student] = InMemoryRepository()
teachers: InMemoryRepository[Teacher] = InMemoryRepository()
teachers.add(Teacher(7, PersonName("Олена", "Ткаченко"), "кафедра програмування"))

# Fix 4: id – ціле число
lecturer = Teacher(1, PersonName("Ігор", "Романюк"), "кафедра системного програмування")
course = Course(1, "Мова C", Credits(4), lecturer)

# Fix 5: назва групи загорнута у value object, який її ще й валідує
newcomer = Student(10, PersonName("Оксана", "Ференц"), GroupCode("ФЕП-31c"))
students.add(newcomer)
print(student_id, average(), len(students), len(teachers), course, newcomer)

$ MYPYPATH=src python -m mypy --strict ../lab4/experiment/type_errors_after.py
Success: no issues found in 1 source file

$ python ../lab4/experiment/type_errors_after.py
1 90.5 1 1 1 Мова C (4 кред., Романюк І.) Ференц Оксана, ФЕП-31c

```

17. Автоматизовані тести

`tests/test_domain.py` — 19 тестів: `value objects` (незмінність, валідація, порядок, тип результату `constrained TypeVar`), рівність сутностей за `id`, ланцюжок `MRO`, `GradeBook` як колекція, правила `Group`, `generic`-репозиторій, `round-trip TypedDict` $\rightleftharpoons$ `Student`, структурна перевірка

Serializable, сервіси через контейнер із RecordingNotifier, взаємозамінність policy, звіти, ієрархія винятків і обидва імпортери. Разом із тестами ЛР1 та ЛР2 — 40 тестів.

```
from dataclasses import FrozenInstanceError

import pytest

from student_manager.domain import (
    AcademicStatus,
    AlreadyEnrolledError,
    Container,
    Course,
    Credits,
    DomainError,
    DuplicateEntityError,
    EntityNotFoundError,
    Grade,
    GradeBook,
    Group,
    GroupCode,
    InMemoryRepository,
    NotEnrolledError,
    PersonName,
    RecordingNotifier,
    Serializable,
    StrictPolicy,
    Student,
    StudentPayload,
    Teacher,
    ThresholdPolicy,
    WrongGroupError,
    validated_points,
)
from student_manager.domain.importing import (
    ArrowFormatter,
    CsvGradeImporter,
    DelimitedParser,
    GradeImporter,
    RangeValidator,
    TableFormatter,
)
from student_manager.exceptions import InvalidGradeError, InvalidGroupError, StudentManagerError

TEACHER = Teacher(1, PersonName("Олена", "Ткаченко"), "кафедра програмування")

def course(course_id: int = 10, title: str = "Професійний Python", credits: int = 4) -> Course:
    return Course(course_id, title, Credits(credits), TEACHER)

def student(student_id: int = 1, group: str = "ФЕП-31с") -> Student:
    return Student(student_id, PersonName("Марта", "Гнатишин"), GroupCode(group))

# ----- value objects
```

```

def test_grade_is_frozen_validated_and_ordered() -> None:
    grade = Grade(95)
    assert grade.points == 95.0 and grade.ects == "A" and grade.is_excellent
    with pytest.raises(FrozenInstanceError):
        grade.points = 10 # type: ignore[misc]
    with pytest.raises(InvalidGradeError):
        Grade(100.5)
    assert Grade(58) < Grade(60) <= Grade(60.0) and Grade(90) == Grade(90.0)
    assert max(Grade(81), Grade(95), Grade(60.5)) == Grade(95)
    assert [g.ects for g in (Grade(59), Grade(60), Grade(70), Grade(75), Grade(85), Grade(90))] ==
["FX", "E", "D", "C", "B", "A"]

```

```

def test_validated_points_keeps_the_type() -> None:
    assert validated_points(90) == 90 and isinstance(validated_points(90), int)
    assert validated_points(90.5) == 90.5
    with pytest.raises(InvalidGradeError):
        validated_points(-1)

```

```

def test_person_name_and_group_code_normalise_and_validate() -> None:
    name = PersonName(" Марта ", "Гнатишин ")
    assert name.full == "Гнатишин Марта" and name.initials == "Гнатишин М."
    with pytest.raises(ValueError):
        PersonName("", "Гнатишин")
    code = GroupCode(" ФЕП-31с ")
    assert (code.faculty, code.stream, str(code)) == ("ФЕП", "31", "ФЕП-31с")
    with pytest.raises(InvalidGroupError):
        GroupCode("31-ФЕП")

```

```

def test_credits_add_and_sum() -> None:
    assert Credits(4) + Credits(3) == Credits(7)
    assert sum((Credits(4), Credits(3)), start=Credits(0)) == Credits(7)
    with pytest.raises(ValueError):
        Credits(-1)

```

----- entities

```

def test_entities_compare_by_id_and_type() -> None:
    a, b = student(1), Student(1, PersonName("Інше", "Ім'я"), GroupCode("ФЕП-32"))
    assert a == b and hash(a) == hash(b) and len({a, b}) == 1
    assert a != student(2)
    assert a != Teacher(1, PersonName("Марта", "Гнатишин"), "кафедра")
    with pytest.raises(ValueError):
        student(0)

```

```

def test_inheritance_chain_and_str() -> None:
    s = student()

```

```

assert isinstance(s, Student) and str(s) == "Гнатишин Марта, ФЕР-31с"
assert str(TEACHER) == "Ткаченко Олена (кафедра програмування)"
assert [cls.__name__ for cls in Student.__mro__[:3]] == ["Student", "Person", "Entity"]

def test_gradebook_is_a_collection() -> None:
    book = GradeBook()
    python, c_lang = course(10), course(11, "Мова С")
    book.enroll(python)
    book.enroll(c_lang)
    book.add(python, Grade(95))
    book.add(python, Grade(85))
    book.add(c_lang, Grade(50))
    assert len(book) == 3 and python in book and course(12) not in book
    assert book[python] == (Grade(95), Grade(85))
    assert [(c.title, g.points) for c, g in book] == [("Професійний Python", 95), ("Професійний
Python", 85), ("Мова С", 50)]
    assert book.average == pytest.approx((95 + 85 + 50) / 3)
    assert book.average_for(python) == 90 and book.earned_credits == Credits(4)
    with pytest.raises(AlreadyEnrolledError):
        book.enroll(python)
    with pytest.raises(NotEnrolledError):
        book.add(course(12), Grade(70))
    with pytest.raises(NotEnrolledError):
        book[course(12)]

def test_empty_gradebook_has_no_average() -> None:
    s = student()
    assert s.average is None and len(s.book) == 0 and s.book.earned_credits == Credits(0)

def test_group_roster_rules() -> None:
    group = Group(1, GroupCode("ФЕР-31с"))
    group.add(student(1))
    assert len(group) == 1 and student(1) in group and group.average is None
    with pytest.raises(DuplicateEntityError):
        group.add(student(1))
    with pytest.raises(WrongGroupError):
        group.add(student(2, "ФЕР-32"))

# ----- repository and DTO

def test_generic_repository() -> None:
    repo = InMemoryRepository[Teacher]([TEACHER], kind="викладач")
    assert len(repo) == 1 and 1 in repo and 2 not in repo and repo.get(2) is None
    assert repo.require(1) is TEACHER
    with pytest.raises(EntityNotFoundError) as info:
        repo.require(2)
    assert "викладач" in str(info.value) and info.value.entity_id == 2
    with pytest.raises(DuplicateEntityError):
        repo.add(TEACHER)
    assert repo.remove(1) is True and repo.remove(1) is False and repo.all() == []

```

```

def test_payload_round_trip_and_optional_email() -> None:
    payload: StudentPayload = {"id": 5, "first_name": "Ірина", "last_name": "Пасічник", "group": "ФЕП-32"}
    s = Student.from_payload(payload)
    assert s.email is None and s.contact == "Пасічник Ірина" and s.to_dict() == payload
    with_email: StudentPayload = {**payload, "email": "i@lnu.edu.ua"}
    assert Student.from_payload(with_email).to_dict() == with_email


def test_serializable_is_structural() -> None:
    assert isinstance(student(), Serializable) and isinstance(Grade(70), Serializable)
    assert not isinstance(TEACHER, Serializable) and not isinstance("text", Serializable)
    assert Grade(70).to_dict() == {"points": 70.0, "ects": "D"}


# ----- services


@pytest.fixture
def app() -> tuple[Container, RecordingNotifier]:
    log = RecordingNotifier()
    container = Container.build(notifier=log)
    container.courses.add(course(10))
    container.courses.add(course(11, "Мова С"))
    return container, log


def test_admit_creates_group_and_notifies(app: tuple[Container, RecordingNotifier]) -> None:
    container, log = app
    group = container.enrollment.admit(student(1))
    container.enrollment.admit(student(2))
    assert len(container.groups) == 1 and len(group) == 2 and str(group.code) == "ФЕП-31с"
    assert log.sent == [("Гнатишин Марта", "Вас зараховано до групи ФЕП-31с")] * 2
    with pytest.raises(DuplicateEntityError):
        container.enrollment.admit(student(1))


def test_enroll_grade_average_and_status(app: tuple[Container, RecordingNotifier]) -> None:
    container, log = app
    container.enrollment.admit(student(1))
    container.enrollment.enroll(1, 10)
    container.enrollment.enroll(1, 11)
    assert container.grading.status_of(1) is AcademicStatus.NO_GRADES
    container.grading.grade(1, 10, 95)
    container.grading.grade(1, 11, 91)
    assert container.grading.average_of(1) == pytest.approx(93)
    assert container.grading.status_of(1) is AcademicStatus.EXCELLENT
    container.grading.grade(1, 11, 40)
    assert log.sent[-1] == ("Гнатишин Марта", "Незадовільна оцінка з «Мова С»: 40.0 (FX)")
    container.courses.add(course(12, "Методи обчислень", 3))
    with pytest.raises(NotEnrolledError):
        container.grading.grade(1, 12, 70)

```

```

with pytest.raises(EntityNotFoundError):
    container.grading.grade(1, 99, 70)

def test_policies_are_interchangeable_strategies() -> None:
    s = student()
    python, c_lang = course(10), course(11, "Moba C")
    for c in (python, c_lang):
        s.enroll(c)
    s.record(python, Grade(95))
    s.record(c_lang, Grade(55))
    assert ThresholdPolicy().status_of(s) is AcademicStatus.GOOD
    assert StrictPolicy().status_of(s) is AcademicStatus.AT_RISK
    assert ThresholdPolicy(excellent=70).status_of(s) is AcademicStatus.EXCELLENT

def test_reporting(app: tuple[Container, RecordingNotifier]) -> None:
    container, _ = app
    for student_id, points in ((1, 95), (2, 70), (3, 95)):
        container.enrollment.admit(Student(student_id, PersonName("Ім'я", f"Прізвище{student_id}"),
        GroupCode("ФЕП-31c")))
        container.enrollment.enroll(student_id, 10)
        container.grading.grade(student_id, 10, points)
    ranking = container.reporting.ranking()
    assert [(s.id, avg) for s, avg in ranking] == [(1, 95.0), (3, 95.0), (2, 70.0)]
    report = container.reporting.student_report(2)
    assert report == {"id": 2, "name": "Прізвище2 Ім'я", "group": "ФЕП-31c", "courses": 1, "grades": 1,
        "average": 70.0, "credits": 4, "status": AcademicStatus.GOOD}
    (group_report,) = container.reporting.group_reports()
    assert group_report == {"code": "ФЕП-31c", "students": 3, "average": pytest.approx(86.67,
    abs=0.01), "best": "Прізвище1 Ім'я"}
    assert [s.id for s in container.reporting.with_status(AcademicStatus.EXCELLENT)] == [1, 3]

def test_domain_errors_are_student_manager_errors() -> None:
    assert issubclass(DomainError, StudentManagerError)
    for error in (EntityNotFoundError, DuplicateEntityError, NotEnrolledError, WrongGroupError):
        assert issubclass(error, DomainError)
    assert issubclass(EntityNotFoundError, LookupError) and issubclass(WrongGroupError, ValueError)

# ----- inheritance vs
# composition

def test_both_importers_agree() -> None:
    lines = ["1,10,95", "", "7,11,55"]
    inherited = CsvGradeImporter().import_lines(lines)
    composed = GradeImporter(DelimitedParser(), RangeValidator(), ArrowFormatter()).import_lines(lines)
    assert inherited == composed == ["1:10 -> 95.0 (A)", "7:11 -> 55.0 (FX)"]
    with pytest.raises(InvalidGradeError):
        CsvGradeImporter().import_lines(["1,10,150"])

```

```

def test_composed_parts_are_testable_alone() -> None:
    assert DelimitedParser(";").parse("1; 10; 95") == {"student_id": 1, "course_id": 10, "points":
95.0}
    assert RangeValidator().validate({"student_id": 1, "course_id": 10, "points": 61}) == Grade(61)
    assert TableFormatter().format({"student_id": 1, "course_id": 10, "points": 61}, Grade(61)) == "
1 | 10 | 61.0 | D | "

===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\user\vscode\uni\python\project-
lab4\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\vscode\uni\python\project-lab4
configfile: pyproject.toml
testpaths: tests
collecting ... collected 40 items

tests/test_analytics.py::test_every_student_has_a_grade_per_discipline PASSED [ 2%]
tests/test_analytics.py::test_unique_groups PASSED [ 5%]
tests/test_analytics.py::test_student_average PASSED [ 7%]
tests/test_analytics.py::test_best_student PASSED [ 10%]
tests/test_analytics.py::test_ranking_is_descending PASSED [ 12%]
tests/test_analytics.py::test_threshold_closure_remembers_minimum PASSED [ 15%]
tests/test_analytics.py::test_above_threshold PASSED [ 17%]
tests/test_analytics.py::test_counts_and_students_per_group PASSED [ 20%]
tests/test_analytics.py::test_nested_grouping PASSED [ 22%]
tests/test_analytics.py::test_group_averages PASSED [ 25%]
tests/test_analytics.py::test_args_and_kwargs PASSED [ 27%]
tests/test_analytics.py::test_filter_and_pipeline PASSED [ 30%]
tests/test_domain.py::test_grade_is_frozen_validated_and_ordered PASSED [ 32%]
tests/test_domain.py::test_validated_points_keeps_the_type PASSED [ 35%]
tests/test_domain.py::test_person_name_and_group_code_normalise_and_validate PASSED [ 37%]
tests/test_domain.py::test_credits_add_and_sum PASSED [ 40%]
tests/test_domain.py::test_entities_compare_by_id_and_type PASSED [ 42%]
tests/test_domain.py::test_inheritance_chain_and_str PASSED [ 45%]
tests/test_domain.py::test_gradebook_is_a_collection PASSED [ 47%]
tests/test_domain.py::test_empty_gradebook_has_no_average PASSED [ 50%]
tests/test_domain.py::test_group_roster_rules PASSED [ 52%]
tests/test_domain.py::test_generic_repository PASSED [ 55%]
tests/test_domain.py::test_payload_round_trip_and_optional_email PASSED [ 57%]
tests/test_domain.py::test_serializable_is_structural PASSED [ 60%]
tests/test_domain.py::test_admit_creates_group_and_notifies PASSED [ 62%]
tests/test_domain.py::test_enroll_grade_average_and_status PASSED [ 65%]
tests/test_domain.py::test_policies_are_interchangeable_strategies PASSED [ 67%]
tests/test_domain.py::test_reporting PASSED [ 70%]
tests/test_domain.py::test_domain_errors_are_student_manager_errors PASSED [ 72%]
tests/test_domain.py::test_both_importers_agree PASSED [ 75%]
tests/test_domain.py::test_composed_parts_are_testable_alone PASSED [ 77%]
tests/test_registry.py::test_by_group_ignores_case_and_spaces PASSED [ 80%]
tests/test_registry.py::test_group_average PASSED [ 82%]
tests/test_registry.py::test_best_overall_and_inside_group PASSED [ 85%]
tests/test_registry.py::test_best_of_unknown_group_raises PASSED [ 87%]
tests/test_registry.py::test_grade_outside_scale_is_rejected PASSED [ 90%]
tests/test_registry.py::test_group_name_is_validated PASSED [ 92%]
tests/test_registry.py::test_duplicate_student_is_rejected PASSED [ 95%]
tests/test_registry.py::test_search_combines_criteria PASSED [ 97%]
tests/test_registry.py::test_rating_is_descending PASSED [100%]

===== 40 passed in 0.33s =====

```

18. Робота з Git

Роботу виконано в окремій гілці lab4, створеній від main, шістьма комітами — по одному на шар моделі: value objects і винятки; сутності; протоколи, репозиторій і DTO; сервіси, policy, notifiers і контейнер; експеримент inheritance/composition; демонстрація, тести, конфігурація туру і версія 0.4.0. Гілку буде злино в main після перевірки.

18.1. Результат git status

```
On branch lab4
nothing to commit, working tree clean
```

18.2. Результат git log --oneline

```
f733ff1 Add domain demo and tests, enable strict mypy config, bump version to 0.4.0
62b9963 Add grade importer in inheritance and composition styles
1515e3a Add application services, status policies, notifiers and DI container
2178140 Add protocols, generic repository and TypedDict payloads
2227c73 Add domain entities: Person hierarchy, Course, Group, GradeBook
d8f7de6 Add domain value objects and exception hierarchy
ec3e1ec Fix group name in README
9770176 Wire analytics demo into CLI, add tests, bump version to 0.2.0
94e46bd Add list vs dict search benchmark
3810ae2 Add analytics: averages, ranking, threshold closure, pipeline
28f41c3 Add collection processors: filtering, grouping, indexing, counting
5dd0a69 Add timing and logging decorators with a bounded call history
6c8b9a6 Add analytics package with per-discipline grade records
ce3e20c Make stdout re-encoding type-safe for mypy --strict
ca3ca1a Add registry tests and README
c01835d Add console menu and application entry point
8502b7c Add StudentRegistry business logic
7726d17 Add Student model with validation and domain exceptions
9cee0a2 Create project skeleton with src-layout
```

19. Відповіді на контрольні запитання

1. Що таке class?

Опис типу об'єктів: які дані вони зберігають (атрибути) і що вміють (методи). Шаблон, за яким створюються об'єкти; у проєкті — Student, Grade, Repository.

2. Що таке object?

Конкретний екземпляр класу зі своїм станом у пам'яті: Grade(95) і Grade(81) — два об'єкти одного класу.

3. Що таке instance?

Те саме, що об'єкт, з наголосом на належності до класу: student є instance класу Student, isinstance(student, Student) повертає True.

4. Для чого __init__?

Ініціалізатор: викликається одразу після створення об'єкта, отримує аргументи конструктора й наповнює атрибути. У Entity.__init__ ще й перевіряє, що id додатний.

5. Що таке instance attribute?

Дані конкретного об'єкта: self.name, self._group. У двох студентів вони різні.

6. Що таке class attribute?

Атрибут, оголошений у тілі класу і спільний для всіх екземплярів: __slots__, константи. Поведінка рівня класу — classmethod, наприклад Student.from_payload.

7. Що таке encapsulation?

Об'єкт сам контролює свій стан: зміни відбуваються через методи, що перевіряють правила (GradeBook.add вимагає попереднього enroll), а не через пряме присвоєння атрибутам.

8. Для чого properties?

Дати атрибутоподібний доступ до обчислюваних або захищених значень: student.average рахується із залікової книжки, student.group читається, але не присвоюється.

9. Чим property відрізняється від public field?

Зовні виглядає як поле, але за ним стоїть метод: можна обчислювати, валідувати, зробити read-only і змінити реалізацію, не зламавши код, який це поле читає.

10. Що таке inheritance?

Створення класу на основі іншого: нащадок отримує атрибути й методи батька та може їх доповнити або перевизначити — Student(Person).

11. Що означає relation is-a?

«Є різновидом»: Student є Person, тому там, де очікують Person, підійде Student. Лише такий зв'язок виправдовує наслідування.

12. Що таке composition?

Клас будує поведінку з інших об'єктів, які зберігає як частини: Student має GradeBook, Course має Teacher, GradeImporter має parser, validator і formatter.

13. Що означає relation has-a?

«Має»: об'єкт володіє іншим об'єктом або посилається на нього як на компонент, а не є його різновидом.

14. Коли composition краща за inheritance?

Коли потрібна поведінка, а не підтип: компоненти можна міняти під час виконання, тестувати окремо й комбінувати. Наслідування жорстко прив'язує до батька і множить підкласи з кожною новою віссю варіації.

15. Для чого `super()`?

Викликати реалізацію батьківського класу з перевизначеного методу: `Student.__init__` через `super()` віддає `id` та ім'я `Person`, `StrictPolicy.status_of` доповнює правило `ThresholdPolicy`, а не копіює його.

16. Що таке `polymorphism`?

Один і той самий виклик працює з об'єктами різних типів: сервіс викликає `notifier.send()`, не знаючи, консоль це чи журнал; `policy.status_of()` — `Threshold` чи `Strict`.

17. Що таке `duck typing`?

Придатність об'єкта визначається наявністю потрібних методів, а не його класом: `RecordingNotifier` ніде не успадковує `Notifier`, але має `send()` — і цього досить.

18. Що таке `ABC`?

Абстрактний базовий клас із модуля `abc`: оголошує обов'язкові методи через `@abstractmethod` і не дає створити екземпляр, поки нащадок їх не реалізує — `Repository`, `AcademicStatusPolicy`.

19. Для чого `abstractmethod`?

Позначити метод, який нащадок зобов'язаний реалізувати; клас із нереалізованими абстрактними методами сам лишається абстрактним.

20. Чи можна створити `instance abstract class`?

Ні: `Repository[Student]()` кидає `TypeError`, поки `add/get/all/remove` не реалізовані. Екземпляр створюється лише від повного нащадка.

21. Що таке `Protocol`?

Клас із `typing`, який описує інтерфейс за формою: будь-який об'єкт із методами потрібної сигнатури сумісний із ним без наслідування.

22. Чим `Protocol` відрізняється від `ABC`?

`ABC` — номінальний контракт: треба успадкувати, перевірка в `runtime` при створенні, може містити спільну реалізацію. `Protocol` — структурний: достатньо збігу сигнатур, перевіряє `type checker`; `isinstance` працює лише з `runtime_checkable`.

23. Що таке `structural typing`?

Сумісність типів за структурою — наявними методами й атрибутами, — а не за іменем чи родоводом. Так працюють Protocol і duck typing.

24. Що таке nominal typing?

Сумісність за явним оголошенням: тип підходить, якщо це той самий клас або його нащадок. Так працюють ABC і звичайне наслідування.

25. Що таке dataclass?

Декоратор, який за анотаціями полів генерує `__init__`, `__repr__`, `__eq__` (за потреби порядок, `hash`, `slots`), щоб не писати шаблонний код.

26. Які methods генерує dataclass?

`__init__`, `__repr__`, `__eq__` завжди; з `order=True` — `__lt__`, `__le__`, `__gt__`, `__ge__`; з `frozen=True` — `__setattr__` і `__delattr__`, які забороняють зміну, та `__hash__`; зі `slots=True` — `__slots__`; також `__match_args__`.

27. Для чого default_factory?

Створювати значення за замовчуванням окремо для кожного екземпляра — `field(default_factory=list)` — щоб об'єкти не ділили один mutable об'єкт.

28. Чому mutable default є проблемою?

Значення за замовчуванням обчислюється один раз при визначенні; список як default стає спільним для всіх об'єктів, і `append` в одному з'являється в усіх.

29. Для чого __post_init__?

Хук після згенерованого `__init__`: валідація й нормалізація полів — `Grade` перевіряє межі, `PersonName` обрізає пробіли.

30. Що означає frozen=True?

Після створення поля не можна змінювати (`FrozenInstanceError`); об'єкт стає hashable і придатний для `set` чи ключа `dict` — те, що потрібно value object.

31. Що означає slots=True?

Клас отримує `__slots__` замість `__dict__`: менше пам'яті, швидший доступ і заборона випадкових нових атрибутів — `student.grup = ...` впаде з `AttributeError`.

32. Що таке dunder method?

Метод із двома підкресленнями з обох боків (`__len__`, `__eq__`), через який Python реалізує вбудовані операції: `len()`, `==`, `for`, `in`, `+`.

33. Для чого `__repr__()`?

Однозначне подання для розробника й відлагодження: `Student(id=1, name='Гнатишин Марта')`; його показує консоль і використовують колекції.

34. Для чого `__str__()`?

Подання для людини: `print(student)` дає «Гнатишин Марта, ФЕП-31с». Якщо `__str__` немає, береться `__repr__`.

35. Для чого `__len__()`?

Щоб працював `len(obj)`: `len(book)` — кількість оцінок, `len(group)` — кількість студентів.

36. Для чого `__iter__()`?

Щоб об'єкт можна було обходити у `for` і передавати в `list()`, `sorted()`, `sum()`: `for course, grade in book`.

37. Для чого `__contains__()`?

Щоб працював оператор `in`: `course in book`, `student in group`, `2 in repository`.

38. Для чого `__call__()`?

Щоб екземпляр можна було викликати як функцію: `obj(x)`. Зручно для стратегій та об'єктів-налаштувань, наприклад `Discount(10)(100)`.

39. Що таке SOLID?

П'ять принципів проєктування об'єктно-орієнтованих систем: єдина відповідальність, відкритість/закритість, підстановка Лісков, розділення інтерфейсів, інверсія залежностей.

40. Що означає SRP?

Клас має одну причину для зміни: `GradeBook` відповідає лише за оцінки, `ReportingService` — лише за звіти, `ConsoleNotifier` — лише за доставку повідомлень.

41. Що означає OCP?

Відкритість для розширення, закритість для змін: нова `policy` чи новий `notifier` — новий клас, а сервіси не редагуються.

42. Що означає LSP?

Нащадок має працювати всюди, де очікують батька, не порушуючи контракту: `StrictPolicy` замість `ThresholdPolicy`, `InMemoryRepository` замість `Repository`.

43. Що означає ISP?

Краще кілька вузьких інтерфейсів, ніж один широкий: `HasId`, `Serializable`, `Notifier` — по одному методу; клієнт залежить лише від того, що справді використовує.

44. Що означає DIP?

Високорівнева логіка залежить від абстракцій, а не від конкретних класів: `GradingService` приймає `Repository[Student]` і `Notifier`, а не `dict` і `print`.

45. Що таке `dependency injection`?

Передача залежностей об'єкту ззовні через конструктор замість створення їх усередині; збирання відбувається в одному місці — `Container.build()`.

46. Чому DI покращує `testability`?

У тест підставляється двійник: `RecordingNotifier` замість консолі, порожній `InMemoryRepository` замість бази — і поведінка сервісу перевіряється ізольовано, без побічних ефектів і патчів.

47. Що таке `type hint`?

Анотація типу параметра, результату чи змінної: `def grade(points: float) -> Grade`. Документує контракт, допомагає IDE і `type checker`.

48. Чи перевіряє Python `type hints` автоматично в `runtime`?

Ні. Анотації зберігаються в `__annotations__`, але інтерпретатор їх не застосовує: `student_id: int = "1"` виконується без помилки — розділ 16.2.

49. Що таке `static type checker`?

Інструмент, який аналізує код за анотаціями до запуску й повідомляє про несумісні типи; поведінку програми не змінює.

50. Для чого `myru`?

Статична перевірка типів Python-коду з командного рядка або CI; у проєкті — `python -m myru src --strict`.

51. Для чого `Pyright`?

Альтернативний `type checker` від Microsoft (основа `PyLance` у VS Code): працює в IDE у реальному часі, перевіряє те саме, подекуди суворіше.

52. Що таке `TypeVar`?

Змінна типу — заповнювач, який `type checker` зв'язує з конкретним типом у місці використання, зберігаючи зв'язок між входом і виходом.

53. Що таке `Generic`?

Базовий клас, що робить клас параметризованим типом: `class Repository(Generic[T])` — один клас для `Student`, `Course` і `Teacher`.

54. Що означає `Generic[T]`?

Клас має параметр типу `T`; при використанні його підставляють: `Repository[Student]` — тоді `add` приймає лише `Student`, а `get` повертає `Student` | `None`.

55. Що таке `bound TypeVar`?

`TypeVar` із верхньою межею: `T = TypeVar("T", bound=HasId)` — підійде будь-який тип, що має `id`, і всередині можна безпечно звертатися до `item.id`.

56. Що таке `constrained TypeVar`?

`TypeVar` із фіксованим переліком типів: `Points = TypeVar("Points", int, float)` — лише `int` або `float`, і результат має той самий тип, що й аргумент.

57. Що таке `TypedDict`?

Опис `dict` із відомими ключами й типами значень для `type checker`; у `runtime` — звичайний `dict` без валідації.

58. Чим `TypedDict` відрізняється від `dataclass`?

`TypedDict` — форма словника: доступ через `["key"]`, немає методів, ідентичності, `__post_init__`. `dataclass` — справжній клас з екземплярами, методами, валідацією, властивостями.

59. Коли доцільно використовувати `TypedDict`?

На межі системи: `JSON` з `API`, рядки `CSV`, `payload` для бази, результати сервісів (`StudentReport`) — там, де дані й так є словником.

60. Чому `external payload` і `domain object` краще розділяти?

`Payload` — ненадійні й неповні дані у форматі зовнішнього світу (`email` може бути відсутній); `domain object` гарантує інваріанти (`GroupCode` валідний, `Grade` у межах). Перетворення в одному місці захищає модель, а зовнішній формат можна змінювати, не чіпаючи сутностей.

20. Висновок

До проєкту «Система обліку студентів» додано пакет `domain` — типізовану об'єктну модель: чотири `frozen value objects` з валідацією, ієрархію сутностей `Entity` → `Person` → `Student` / `Teacher` із рівністю за `id`, `Course` і `Group`, власну колекцію `GradeBook` з чотирма `dunder`-методами, ієрархію винятків поверх `StudentManagerError`, три `Protocol` і два `ABC`, `generic`-репозиторій на `bound TypeVar`, чотири `TypedDict` на межі із зовнішнім світом, три сервіси з `dependency injection`, стратегію `academic status` і ручний `DI`-контейнер. Експерименти показали, що `DI` робить сервіс тестованим без патчів, а композиція перестає множити класи там, де наслідування починає; туру

у strict-режимі знайшов п'ять навмисних помилок, які інтерпретатор пропустив би мовчки, і дві справжні у моєму коді. Пакет покрито 19 тестами (40 разом із попередніми), проходить туру `--strict` на 26 файлах і зафіксований шістьма комітами в гілці `lab4`. Наступні роботи отримують готовий шов для бази даних — `Repository[T]` — і готовий формат для API — `TypedDict`-звіти.